المطعم: حل الواجب

لكل طلب من الطلبات الخمسة: **كيف نفكر**، ثم **الكود**، ثم **الناتج**، ثم جواب السؤال. الكود كاملاً في مجلد `solution_code` (والمدخلات في `input.txt` فيه).

1 الشوربة (Soup): صنف جديد يقبل التخفيض

كيف نفكر؟

- هل الشوربة **هي** صنف في القائمة؟ نعم ← `extends MenuItem`.
- هل **تستطيع** أن يُخفَّض سعرها؟ نعم ← `implements Discountable`.
- ما معلومتها الخاصة؟ الحجم بالمل ← خاصية `ml` في `Soup` فقط.
- ماذا نكتب؟ البنّاء و`toString()` و`priceAfterDiscount()` (20%).

Soup.java

```java
package restaurant;

public class Soup extends MenuItem implements Discountable {
    private int ml;

    public Soup(String name, int price, int ml) {
        super(name, price);
        this.ml = ml;
    }

    @Override
    public String toString() {
        return "Soup: " + getName() + " (" + ml + " ml, " + getPrice() + "$)";
    }

    @Override
    public int priceAfterDiscount() {
        return getPrice() * 80 / 100;
    }
}
```

هو صنف + يُخفَّض

يصف نفسه بطريقته

تخفيض 20%

وفي `Main` أضفنا النوع الرابع في القائمة، ولم نغيّر أي حلقة (الحلقات تتعامل مع `MenuItem` و`Discountable`):

```
======== Menu ========
Meal: Pizza (300 g, 800$)
Drink: Juice (250 ml, 150$)
Dessert: Cake (400 cal, 300$)
Drink: Cola (330 ml, 100$)
Soup: Lentil (300 ml, 100$)
...
======== Discounts ========
Juice: 150$ -> 135$
Cake: 300$ -> 240$
Cola: 100$ -> 90$
Lentil: 100$ -> 80$
No discount: 1
```

2 المحاسب (Cashier): موظف جديد

كيف نفكر؟

- هل المحاسب **هو** موظف؟ نعم ← `extends Employee` .
- الراتب والتحقق منه والواجهة `Payable` كلها **عند الأب** ← لا نكتبها.
- نكتب فقط: البنّاء و `toString()` .

Cashier.java

```java
package restaurant;

public class Cashier extends Employee {

    public Cashier(String name, String phone, String email, int salary) {
        super(name, phone, email, salary);
    }

    @Override
    public String toString() {
        return "Cashier: " + getName() + " (" + getPhone() + ", " + getEmail() + ", salary " + salary() + "$)";
    }
}
```

يرث الراتب و Payable من الأب

```
======== Salaries ========
Hana: 5000$
Rami: 3500$
Sara: 4000$
Total salaries: 12500$
```

سؤال: لماذا لم نكتب `implements Payable` في `Cashier` ؟

لأن `Employee` كُتب فيه `implements Payable` ، و `Cashier` يرث من `Employee` . **الابن يرث من أبيه الخصائص والتوابع والواجهات معاً.** لذلك دخل راتب سارة ($4000) في المجموع تلقائياً: 5000 + 3500 + 4000 = 12500.

3 رفض الكمية الصفر أو السالبة (Encapsulation)

كيف نفكر؟

- من **يبني** عنصر الطلب؟ `Order` (في `addItem`). إذن هو **الحارس** الذي يتحقق.
- لماذا لا نتحقق في `main` ؟ لأن أي مكان آخر ينادي `addItem` سيتجاوز التحقق. الصف الذي يملك المعلومة هو الذي يحميها.
- ماذا نفعل إن كانت الكمية خاطئة؟ نطبع `Invalid quantity` ونخرج **قبل** أن نبني العنصر.

Order.java

```java
package restaurant;

import java.util.ArrayList;

public class Order {
    private Customer customer;
    private ArrayList<OrderItem> items = new ArrayList<>();

    public Order(Customer customer) {
        this.customer = customer;
    }

    public Customer getCustomer() {
        return customer;
    }

    public void addItem(MenuItem menuItem, int quantity) {
        if (quantity <= 0) {
            System.out.println("Invalid quantity");
            return;
        }
        items.add(new OrderItem(menuItem, quantity));
    }

    public ArrayList<OrderItem> getItems() {
        return items;
    }

    public int total() {
        int sum = 0;
        for (OrderItem orderItem : items) {
            sum += orderItem.subtotal();
        }
        return sum;
    }
}
```

الحارس: هل الكمية صالحة؟

رسالة للمستخدم

نخرج دون أن نبني العنصر

لا نصل هنا إلا بكمية صحيحة

في المدخلات: العنصر الثالث كميته 0، فلم يُضَف والمجموع لم يتغير:

```
Invalid quantity
Item number (1 to 5):
Quantity:
Pizza x2 = 1600$
Juice x1 = 150$
Lentil x1 = 100$
Total: 1850$
======== Payment and delivery ========
Rami delivers to Nour at Mazzeh (call 0933111222) an order of 1850$
```

4 الواجهة Vegetarian (نباتي)

كيف نفكر؟

- ما الذي يشترك فيه المشروب والحلوى؟ يستطيعان الإجابة: "هل أنا نباتي؟" ← **عقد** ← واجهة.
- لماذا ليست في الأب `MenuItem`؟ لأن `Meal` لا ينفّذها (قد يكون فيها لحم) ولا نريد أن نجبرها.
- كيف نستعملها؟ `instanceof Vegetarian` كما فعلنا مع `Discountable`.

Vegetarian.java

```java
package restaurant;

public interface Vegetarian {
    boolean isVegetarian();
}
```

وعد: تابع بلا جسم

Drink.java

```java
package restaurant;

public class Drink extends MenuItem implements Discountable, Vegetar
    private int ml;

    public Drink(String name, int price, int ml) {
        super(name, price);
        this.ml = ml;
    }

    @Override
    public String toString() {
        return "Drink: " + getName() + " (" + ml + " ml, " + getPrice() + "$)";
    }

    @Override
    public boolean isVegetarian() {
        return true;
    }

    @Override
    public int priceAfterDiscount() {
        return getPrice() * 90 / 100;
    }
}
```

واجهتان معاً: مسموح

ينفّذ بند Vegetarian

وفي `Main` حلقة على القائمة:

```java
for (MenuItem item : menu) {
    if (item instanceof Vegetarian) {
        System.out.println("Vegetarian: " + item.getName());
    }
}
```

```
======== Vegetarian ========
Vegetarian: Juice
Vegetarian: Cake
Vegetarian: Cola
```

لاحظ: الوجبة (Pizza) والشوربة (Lentil) لم تظهرا لأنهما لا تنفّذان `Vegetarian` . ويمكنك جعل الشوربة تنفّذها إن أردت.

5 الواجهة Tippable (بقشيش)

كيف نفكر؟

- من يأخذ البقشيش؟ الشيف والموصّل، **وليس الزبون**.
- هل هما "نفس النوع"؟ كلاهما موظف، لكن **المحاسب أيضاً موظف ولا يأخذ بقشيشاً**. فلا نضعها في `Employee` بل واجهة ينفّذها الاثنان فقط.
- الفائدة: `ArrayList<Tippable>` تحمل الشيف والموصّل، وحلقة واحدة تعطيهما.

Tippable.java

```java
package restaurant;

public interface Tippable {
    void tip(int amount);
}
```

وعد: يستلم بقشيشاً

Chef.java

```java
package restaurant;

public class Chef extends Employee implements Tippable {

    public Chef(String name, String phone, String email, int salary) {
        super(name, phone, email, salary);
    }

    @Override
    public void tip(int amount) {
        System.out.println(getName() + " got a tip of " + amount + "$");
    }

    @Override
    public String toString() {
        return "Chef: " + getName() + " (" + getPhone() + ", " + getEmail() + ", salary " + salary() + "$)";
    }
}
```

هو موظف + يُعطى بقشيشاً

ينفّذ بند Tippable

وفي `Main` :

```
ArrayList<Tippable> tippables = new ArrayList<>();
for (Person person : people) {
    if (person instanceof Tippable) {
        tippables.add((Tippable) person);
    }
}
for (Tippable t : tippables) {
    t.tip(10);
}
```

```
======== Tips ========
Hana got a tip of 10$
Rami got a tip of 10$
```

سؤال: لماذا لا يصلح أن نكتب `tip` داخل `Employee` ؟

لأن ليس كل موظف يأخذ بقشيشاً. لو وضعناه في `Employee` لورثه `Cashier` أيضاً (وسارة محاسبة في مثالنا)، وصار المحاسب يستلم بقشيشاً بلا معنى. الواجهة تجعل الشيف والموصّل فقط يلتزمان بها. لاحظ في الناتج: **سارة لم تظهر في قسم Tips.**

6 تشغيل كامل للحل

الأخضر ما يكتبه المستخدم بالكيبورد:

```
How many menu items?
5
Item 1: 1 = Meal, 2 = Drink, 3 = Dessert, 4 = Soup
1
Name:
Pizza
Price:
800
Grams:
300
Item 2: 1 = Meal, 2 = Drink, 3 = Dessert, 4 = Soup
2
Name:
Juice
Price:
150
Ml:
250
Item 3: 1 = Meal, 2 = Drink, 3 = Dessert, 4 = Soup
3
Name:
Cake
Price:
300
Calories:
400
Item 4: 1 = Meal, 2 = Drink, 3 = Dessert, 4 = Soup
2
Name:
Cola
Price:
100
Ml:
330
Item 5: 1 = Meal, 2 = Drink, 3 = Dessert, 4 = Soup
4
Name:
Lentil
```

```
Price:
100
Ml:
300
How many people?
5
Person 1: 1 = Customer, 2 = Chef, 3 = Driver, 4 = Cashier
1
Name:
Nour
Phone:
0933111222
Email:
nour@mail.com
Address:
Mazzeh
Person 2: 1 = Customer, 2 = Chef, 3 = Driver, 4 = Cashier
2
Name:
Hana
Phone:
0944333444
Email:
hana@mail.com
Salary:
5000
Person 3: 1 = Customer, 2 = Chef, 3 = Driver, 4 = Cashier
3
Name:
Rami
Phone:
0955666777
Email:
rami.mail.com
Salary:
3500
Person 4: 1 = Customer, 2 = Chef, 3 = Driver, 4 = Cashier
4
Name:
Sara
Phone:
0966555444
Email:
sara@mail.com
Salary:
4000
Person 5: 1 = Customer, 2 = Chef, 3 = Driver, 4 = Cashier
1
Name:
Omar
Phone:
0966888999
Email:
omar@mail.com
Address:
Kafarsouseh
======== Menu ========
Meal: Pizza (300 g, 800$)
Drink: Juice (250 ml, 150$)
Dessert: Cake (400 cal, 300$)
Drink: Cola (330 ml, 100$)
Soup: Lentil (300 ml, 100$)
======== People ========
Customer: Nour (0933111222, nour@mail.com, Mazzeh, 0 points)
Chef: Hana (0944333444, hana@mail.com, salary 5000$)
Driver: Rami (0955666777, no email, salary 3500$)
Cashier: Sara (0966555444, sara@mail.com, salary 4000$)
Customer: Omar (0966888999, omar@mail.com, Kafarsouseh, 0 points)
```

```
======== Discounts ========
Juice: 150$ -> 135$
Cake: 300$ -> 240$
Cola: 100$ -> 90$
Lentil: 100$ -> 80$
No discount: 1

======== Vegetarian ========
Vegetarian: Juice
Vegetarian: Cake
Vegetarian: Cola

======== Salaries ========
Hana: 5000$
Rami: 3500$
Sara: 4000$
Total salaries: 12500$

======== Tips ========
Hana got a tip of 10$
Rami got a tip of 10$

======== Order ========
How many order items?
4
Item number (1 to 5):
1
Quantity:
2
Item number (1 to 5):
2
Quantity:
1
Item number (1 to 5):
3
Quantity:
0
Invalid quantity
Item number (1 to 5):
5
Quantity:
1
Pizza x2 = 1600$
Juice x1 = 150$
Lentil x1 = 100$
Total: 1850$

======== Payment and delivery ========
Rami delivers to Nour at Mazzeh (call 0933111222) an order of 1850$
Nour pays 1850$
Nour has 18 points, VIP: false
```

الخلاصة: كل طلب من الخمسة أضفنا فيه **صفاً أو واجهة** دون أن نغيّر الصفوف القديمة إلا في أماكن صغيرة. وهذا هو معنى أن التصميم جيد: يتسع للجديد دون أن ينهار.